



ROADMAP DE DÉVELOPPEMENT

Zümm

Planification Scrum & pipeline DevOps

Projet	Application de gestion apicole (mini-projet Java)
Type	Roadmap de développement (Scrum + DevOps)
Version	1.0
Date	16 juillet 2026
Référence	Cahier des charges Zümm v1.0 (13 juillet 2026)
Cadence	1 sprint de cadrage + 18 sprints de 14 jours — 14/07/2026 → 26/04/2027

19 epics — 80 user stories — 582 story points

Document conforme au plan : Méthodologie / Product Backlog / Sprints /
Risques & charge / Pipeline DevOps / Plan de releases / Monitoring & observabilité

Table des matières

Historique des versions	4
1 Méthodologie : Scrum + DevOps	5
1.1 Objet du document	5
1.2 Cadre Scrum	5
1.2.1 Rôles	5
1.2.2 Cérémonies	5
1.2.3 Definition of Ready (DoR)	6
1.2.4 Definition of Done (DoD)	6
1.3 Principes DevOps	6
2 Product Backlog	7
2.1 Vue d'ensemble des epics	8
2.2 EPIC-001 — Gestion des entités métier (CRUD)	8
2.3 EPIC-002 — Planification et suivi des visites	9
2.4 EPIC-003 — Tableaux de bord et visualisation	9
2.5 EPIC-004 — Indicateurs et capteurs (préparation)	9
2.6 EPIC-005 — Authentification et sécurité	10
2.7 EPIC-006 — Internationalisation et configuration	10
2.8 EPIC-007 — Service web API tierce	10
2.9 EPIC-008 — Fonctionnalités avancées (inspiration marché)	10
2.10 EPIC-009 — Intelligence artificielle (préparation)	11
2.11 EPIC-010 — Tests, qualité et documentation	11
2.12 EPIC-011 — Exploitation et restitution	11
2.13 EPIC-012 — Intelligence spatiale (PostGIS)	12
2.14 EPIC-013 — Robustesse et ergonomie du front	12
2.15 EPIC-014 — Durcissement de la sécurité	12
2.16 EPIC-015 — PWA déployable et restitution visuelle	13
2.17 EPIC-016 — Fiabilité de la synchronisation et tenue à l'échelle	13
2.18 EPIC-017 — Conformité réglementaire du miel	14
2.19 EPIC-018 — Modèle d'autorisation complet	14
2.20 EPIC-019 — Dette technique et contrat vérifié	14
3 Planification des sprints	16
3.1 Calendrier général	16

3.2	Synthèse des sprints	18
3.3	Recommandations d'équilibrage et de pilotage	18
3.3.1	Arbitrage retenu	19
3.3.2	Ordre des dépendances	20
3.3.3	Pilotage	20
3.3.4	Recommandations techniques transversales	20
3.4	Sprint 1 — Fondation : CRUD & configuration	21
3.4.1	Plan d'exécution	21
3.5	Sprint 2 — Ruche, i18n & socle qualité	22
3.5.1	Plan d'exécution	22
3.6	Sprint 3 — Visites & rapports	23
3.6.1	Plan d'exécution	24
3.7	Sprint 4 — Hors-ligne, authentification & RBAC	24
3.7.1	Plan d'exécution	25
3.8	Sprint 5 — Tableaux de bord & pilotage	26
3.8.1	Plan d'exécution	26
3.9	Sprint 6 — Synthèse, capteurs & API	27
3.9.1	Plan d'exécution	27
3.10	Sprint 7 — Fonctions avancées & détection d'anomalie	28
3.10.1	Plan d'exécution	28
3.11	Sprint 8 — Qualité & livraison	29
3.11.1	Plan d'exécution	29
3.12	Sprint 9 — Exploitation & durcissement	30
3.13	Sprint 10 — Intelligence spatiale & remise à niveau	30
3.14	Sprint 11 — Fiabilité de la session et du front	31
3.15	Sprint 12 — Durcissement de la sécurité	32
3.16	Sprint 13 — PWA livrable, graphiques et carte	33
3.17	Sprint 14 — Idempotence, index et conformité miel	34
3.18	Sprint 15 — Ressources de langue externalisées	35
3.19	Sprint 16 — Sessions sans jeton et portée par affectation	35
3.20	Sprint 17 — Dettes techniques	36
3.21	Sprint 18 — Les deux dernières dettes	37
3.22	Suivi en cours de sprint	38
4	Gestion des risques et de la charge	39
4.1	Registre des risques	39
4.2	Charge estimée par sprint	40
4.3	Suivi de la vélocité	41
5	Pipeline DevOps (CI/CD)	42
5.1	Environnements	42
5.2	Vue d'ensemble du pipeline	42
5.3	Détail des étapes	43
5.4	Infrastructure as Code	43
5.5	Mise en place progressive	43

5.6	Stratégie de branches Git	44
6	Plan de releases	45
6.1	Synthèse	45
6.2	v0.1.0 — MVP : Fondation	45
6.3	v0.2.0 — Visites & collaboration	46
6.4	v0.3.0 — Analytics & dashboards	46
6.5	v1.0.0 — Release Candidate	46
6.6	Après la v1.0.0 — cinq versions de consolidation	46
7	Monitoring & observabilité	48
7.1	Dashboards Grafana	48
7.1.1	Zümm — Vue d'ensemble (métier)	48
7.1.2	Zümm — Performance (technique)	48
7.1.3	Zümm — Sécurité	48
7.2	Règles d'alerte	49
7.3	Gestion des logs	49

Historique des versions

Version	Date	Auteur	Modifications
1.0	16 juillet 2026	Équipe projet	Consolidation LaTeX du dossier <code>zumm_scrum_devops_roadmap</code> : backlog, sprints, pipeline CI/CD, releases et monitoring. Correction des totaux de points des sprints 5, 7 et 8.

CHAPITRE 1

Méthodologie : Scrum + DevOps

1.1 Objet du document

Ce document constitue la **roadmap de développement** du projet Zümm. Il traduit le cahier des charges (v1.0, 13 juillet 2026) en un plan d'exécution opérationnel : un *product backlog* priorisé, une planification en **18 sprints** de deux semaines avec plans d'exécution détaillés, un registre des risques coté, une chaîne d'intégration et de déploiement continu (CI/CD), un plan de releases sémantiques et un dispositif de monitoring.

Chiffres clés. 19 epics — 80 user stories — 582 story points — 1 sprint de cadrage + 18 sprints de 14 jours — 4 releases (v0.1.0 → v1.0.0) — période : 14 juillet 2026 → 26 avril 2027.

1.2 Cadre Scrum

1.2.1 Rôles

Rôle	Responsabilité
Product Owner	Priorise le backlog, valide les critères d'acceptation, accepte les livrables en sprint review.
Scrum Master	Garant du cadre, lève les obstacles, anime les cérémonies.
Équipe de développement	Conçoit, développe, teste et livre l'incrément de chaque sprint.

1.2.2 Cérémonies

Chaque sprint suit le même rythme de cérémonies :

Cérémonie	Cadence / durée	Objectif
Sprint Planning	Jour 1, 09h00–13h00 (4 h)	Sélection des stories et engagement de l'équipe.
Daily Scrum	Tous les jours, 09h15 (15 min)	Synchronisation, détection des blocages.
Sprint Review	Dernier jour, 14h00–16h00 (2 h)	Démonstration de l'incrément au Product Owner.
Sprint Retrospective	Dernier jour, 16h00–17h00 (1 h)	Amélioration continue du processus.

1.2.3 Definition of Ready (DoR)

Une story ne peut entrer dans un sprint que si :

1. elle a un titre clair et une description ;
2. ses critères d'acceptation sont définis ;
3. ses dépendances sont identifiées ;
4. elle est estimée en points ;
5. le Product Owner a validé sa priorité.

1.2.4 Definition of Done (DoD)

Une story n'est terminée que si :

1. le code est développé et revu (*peer review*) ;
2. les tests unitaires passent (couverture $\geq 70\%$) ;
3. les tests d'intégration passent ;
4. la documentation est mise à jour ;
5. l'incrément est déployé sur l'environnement de staging ;
6. le Product Owner a validé la story.

1.3 Principes DevOps

La démarche DevOps complète Scrum sur l'axe technique :

- **Intégration continue** : chaque *push* déclenche build, tests et analyse qualité (chapitre 5) ;
- **Déploiement continu** : livraison automatique en staging, déploiement en production sur validation manuelle (*approval gate*) ;
- **Infrastructure as Code** : Docker, docker-compose, manifestes Kubernetes optionnels ;
- **Observabilité** : logs centralisés, métriques Prometheus, dashboards Grafana, alerting (chapitre 7) ;
- **Boucle de rétroaction** : les métriques de production alimentent la priorisation du backlog.

Note de cohérence. Le plan initial couvrait 8 sprints pour 304 points. Le périmètre s'est étendu au fil des revues — durcissement de la sécurité, PWA livrable, conformité miel, puis solde des dettes — pour atteindre **18 sprints de construction et 582 points**. La somme des **story_points** des dix-neuf sprints de **sprints.json** égale le total du *product backlog*, et les trois représentations — product backlog, fichiers opérationnels et présent document — sont alignées.

Réserve sur la vitesse annoncée. Les 582 points comptent les 37 points du sprint 8, dont **21 relèvent de livrables documentaires** (US-039 diagrammes UML, US-040 rapport et soutenance) qui ne sont pas produits : la vitesse *applicative* réellement livrée est donc de **561 points**. Le compter autrement serait surévaluer le livrable.

CHAPITRE 2

Product Backlog

Le backlog est structuré en **19 epics** couvrant l'intégralité du périmètre du cahier des charges, pour un total de **80 user stories** et **582 story points** (suite de Fibonacci : 1, 2, 3, 5, 8, 13, 21).

Les epics 011 et 012 ont été ouverts après la livraison des dix premiers : le produit avait dépassé le périmètre inscrit au backlog, que la roadmap publiée sous-déclarait d'un sprint entier. L'epic 013 est né d'un audit du front mené après le SPRINT-10, qui a mis au jour une session expirant en cinq minutes sans rafraîchissement possible.

Les epics 014 à 019 ne viennent pas du cahier des charges, et c'est ce qui les distingue des treize premiers. Ils viennent de ce que le produit livré a révélé une fois confronté à des conditions réelles : une revue d'architecture qui trouve six écarts entre la sécurité documentée et la sécurité effective (014) ; un front complet mais servi par personne (015) ; un scénario de terrain qui fait apparaître une perte de données silencieuse (016) ; une échéance réglementaire (017) ; une isolation qui séparait les exploitations mais pas les agents (018) ; et une liste de dettes qu'il fallait finir par vider (019).

Cette proportion — 184 points sur 582, soit **30 %** du backlog — n'est pas un dérapage : elle mesure ce qu'un cahier des charges ne peut pas prévoir, et que seule la mise en service révèle.

2.1 Vue d'ensemble des epics

Epic	Intitulé	Priorité	Source CdC	Points
EPIC-001	Gestion des entités métier (CRUD)	Haute	§4.1, §5.2, §7.1	44
EPIC-002	Planification et suivi des visites	Haute	§4.2, §6.1, §6.2	44
EPIC-003	Tableaux de bord et visualisation	Haute	§4.3	42
EPIC-004	Indicateurs et capteurs (préparation)	Moyenne	§4.4	29
EPIC-005	Authentification et sécurité	Haute	§8.2, §8.3, Ann. I	26
EPIC-006	Internationalisation et configuration	Haute	§8.2, §8.3	18
EPIC-007	Service web API tierce	Moyenne	§6.5, §8.2	10
EPIC-008	Fonctionnalités avancées (marché)	Moyenne	§4.6, §4.7	36
EPIC-009	Intelligence artificielle (préparation)	Moyenne	Annexe H	21
EPIC-010	Tests, qualité et documentation	Haute	Chap. 10, 11, Ann. F	52
EPIC-011	Exploitation et restitution	Moyenne	§4.3, §6.5, Ann. G	26
EPIC-012	Intelligence spatiale (PostGIS)	Moyenne	§3.5, Annexe B	21
EPIC-013	Robustesse et ergonomie du front	Haute	§8.2, §8.3, Ann. I	29
EPIC-014	Durcissement de la sécurité	Critique	Annexe G, §6.4	34
EPIC-015	PWA déployable et restitution visuelle	Haute	§8.1, Annexe B	32
EPIC-016	Fiabilité et tenue à l'échelle	Critique	§7.3, Annexe D	21
EPIC-017	Conformité réglementaire du miel	Critique	§3.4, Annexe F	13
EPIC-018	Modèle d'autorisation complet	Critique	Annexe G	42
EPIC-019	Dette technique et contrat vérifié	Haute	Annexe B	42
Total				582

2.2 EPIC-001 — Gestion des entités métier (CRUD)

Opérations CRUD sur fermiers, fermes, sites, ruches, corps/hausses, cadres et agents.

ID	Story	Pts	Priorité	Critères d'acceptation
US-001	CRUD Fermier	5	Haute	CRUD complet avec validation
US-002	CRUD Ferme	5	Haute	Liaison fermier-ferme
US-003	CRUD Site (avec géolocalisation)	8	Haute	Lat/long/altitude + Post-GIS
US-004	CRUD Ruche avec composition	13	Haute	1 corps + 0–5 hausses + 1–10 cadres
US-005	CRUD Agent avec rôles	5	Haute	Rôles : apiculteur, superviseur, responsable, admin
US-006	Contraintes de composition (métier)	8	Haute	Contraintes CHECK SQL + validation Java

2.3 EPIC-002 — Planification et suivi des visites

Planification, approbation, réalisation et rapport de visite.

ID	Story	Pts	Priorité	Critères d'acceptation
US-007	Planifier une visite	8	Haute	Date, raison, actions prévues
US-008	Approuver/refuser un planning	5	Haute	Workflow d'approbation superviseur
US-009	Réaliser une visite et remplir le rapport	13	Haute	Date, heure, durée, constatations, évaluations 1-3
US-010	Ajouter des photos au rapport	5	Haute	Upload multi-photos
US-011	Mode hors-ligne et synchronisation différée	13	Haute	PWA + Service Worker + IndexedDB

2.4 EPIC-003 — Tableaux de bord et visualisation

Calendrier, production, alertes, synthèse ROI.

ID	Story	Pts	Priorité	Critères d'acceptation
US-012	Calendrier matrice agents × ruches	13	Haute	Filtres mois/semaine/saison/année + pop-up
US-013	Tableau de bord production	8	Haute	Seuils paramétrables + collecte/extension
US-014	Tableau de bord alertes sanitaires	8	Haute	Baisse anormale détectée + inspection
US-015	Tableau de bord synthèse et ROI	13	Moyenne	Graphiques production, interventions, ROI

2.5 EPIC-004 — Indicateurs et capteurs (préparation)

Modèle de données ouvert pour l'intégration future des capteurs.

ID	Story	Pts	Priorité	Critères d'acceptation
US-016	Modèle de données Mesure	8	Moyenne	Horodatage, géolocalisation, unités paramétrables
US-017	Interface ingestion REST/MQTT	8	Moyenne	POST <code>/api/mesures</code> + topic MQTT
US-018	Seuils paramétrables et logique d'alerte	8	Moyenne	Anti-rebond / hystérésis
US-019	Conversion d'unités hétérogènes	5	Moyenne	Patron <i>Strategy</i>

2.6 EPIC-005 — Authentication et sécurité

Keycloak (OIDC, fédération Google), RBAC, TLS/X.509.

ID	Story	Pts	Priorité	Critères d'acceptation
US-020	Authentication OAuth 2.0 Google	8	Haute	Flot <i>Authorization Code</i> + JWT
US-021	Authentication locale (repli)	5	Haute	Repli si OAuth indisponible
US-022	Contrôle d'accès RBAC	8	Haute	Matrice 6 profils × 15 fonctions
US-023	Chiffrement TLS 1.3 / X.509	5	Haute	HTTPS forcé + certificats

2.7 EPIC-006 — Internationalisation et configuration

i18n FR/EN/AR, `ConfigZumm.ini`.

ID	Story	Pts	Priorité	Critères d'acceptation
US-024	Internationalisation (FR/EN/AR)	8	Haute	RTL arabe + extensible
US-025	Configuration <code>ConfigZumm.ini</code>	5	Haute	Seuils, unités, modules sans recompilation
US-053	Formatage localisé des dates et nombres	5	Haute	Intl par locale, erreurs traduites

2.8 EPIC-007 — Service web API tierce

Exposition d'une API pour applications externes.

ID	Story	Pts	Priorité	Critères d'acceptation
US-026	Service web REST <code>getZummHoneyActualQuantity</code>	5	Moyenne	REST + contrat OpenAPI 3
US-027	Export CSV/TXT	5	Moyenne	Données maîtrisées par l'utilisateur

2.9 EPIC-008 — Fonctionnalités avancées (inspiration marché)

Fonctions complémentaires inspirées de HiveTracks, Apiary Book et BeePlus.

ID	Story	Pts	Priorité	Critères d'acceptation
US-028	Photos d'inspection	5	Haute	Attachées au rapport
US-029	Contexte météo local	5	Moyenne	API météo par géolocalisation
US-030	Carte et rayons de butinage	8	Moyenne	MapLibre GL + OpenStreetMap + cercles 1/2/3 km
US-031	Liste de tâches et rappels	5	Haute	Calendrier de rappels
US-032	Suivi de la reine	5	Haute	Historique du statut de la reine
US-033	Récolte et QR code	8	Moyenne	QR par lot + traçabilité

2.10 EPIC-009 — Intelligence artificielle (préparation)

Détection d'anomalie adaptative, pipeline IA.

ID	Story	Pts	Priorité	Critères d'acceptation
US-034	Détection d'anomalie adaptative (EWMA)	13	Moyenne	Ligne de base par ruche + z-score
US-035	Interface microservice IA Python	8	Moyenne	REST/JSON découplé

2.11 EPIC-010 — Tests, qualité et documentation

Plan de tests, UML, rapport, poster.

ID	Story	Pts	Priorité	Critères d'acceptation
US-036	Tests unitaires JUnit 5 + Mockito	8	Haute	70 % de couverture métier
US-037	Tests d'intégration (Testcontainers)	5	Haute	Tests <i>in-container</i>
US-038	Tests de charge k6	5	Moyenne	p95 < 500 ms, erreurs < 1 %
US-039	Diagrammes UML complets	13	Haute	10 diagrammes requis
US-040	Rapport + poster + présentation	8	Haute	Livrables de soutenance
US-048	Alignement du backlog et de la roadmap	5	Haute	Backlog et chapitres en phase avec le livré
US-049	Socle de tests et de lint du front	8	Haute	Vitest, ESLint et Prettier joués par la CI

2.12 EPIC-011 — Exploitation et restitution

Notifications d'alerte, restitution documentaire, traçabilité des actions et anticipation de la récolte.

ID	Story	Pts	Priorité	Critères d'acceptation
US-041	Notifications e-mail des alertes	5	Moyenne	Envoi sur franchissement de seuil, désactivable
US-042	Prévision de récolte (poids)	8	Moyenne	Régression linéaire + projection à 7 jours
US-043	Journal d'audit	8	Haute	Aspect AOP, table dédiée avec RLS
US-044	Rapport de visite au format PDF	5	Moyenne	Téléchargement conforme à la charte

2.13 EPIC-012 — Intelligence spatiale (PostGIS)

Exploitation de la base spatiale au-delà de l'affichage : regroupement des sites, voisinage et ordonnancement des déplacements.

ID	Story	Pts	Priorité	Critères d'acceptation
US-045	Regroupement spatial des sites	8	Moyenne	ST_ClusterDBSCAN en base, isolés conservés
US-046	Distances et plus proches voisins	5	Moyenne	Parcours d'index KNN, distance géodésique
US-047	Ordre de tournée optimisé	8	Moyenne	Plus proche voisin + 2-opt, ordre indicatif

2.14 EPIC-013 — Robustesse et ergonomie du front

Session durable, navigation adressable, listes paginées et dialogues cohérents avec le *design system*. Cet epic est né d'un audit du front mené après le SPRINT-10.

ID	Story	Pts	Priorité	Critères d'acceptation
US-050	Rafraîchissement du jeton OI DC	8	Haute	Renouvellement avant expiration, saisie préservée
US-051	Navigation adressable par URL	8	Haute	Une route par écran, retour navigateur fonctionnel
US-052	Pagination des listes	8	Haute	page/taille serveur et client
US-054	Dialogues du <i>design system</i>	5	Moyenne	Plus aucun dialogue natif, clavier et piège de focus

2.15 EPIC-014 — Durcissement de la sécurité

Fermer les écarts entre la sécurité *documentée* et la sécurité *réelle*. Épic ouvert à la suite d'une revue d'architecture : six garanties annoncées au cahier reposaient sur une configuration absente, un contrôle manquant ou une règle jamais appliquée. Aucun de ces écarts n'était visible en test.

ID	Story	Pts	Priorité	Critères d'acceptation
US-058	Claim de tenant garanti, refus explicite	5	Critique	Mapper dans les <i>deux</i> realms ; 403, jamais une liste vide
US-059	Validation d'audience du jeton	3	Critique	Un jeton d'un autre client du royaume est refusé
US-060	Confidentialité des positions	8	Critique	Arrondi selon le rôle, altitude masquée, voisins dégradés
US-061	RBAC en refus par défaut	8	Critique	Un jeton sans rôle métier n'atteint aucun endpoint
US-062	En-têtes de sécurité du proxy	5	Haute	CSP sans <code>unsafe-inline</code> , zone de débit dédiée
US-063	Portes de sécurité bloquantes	5	Haute	SBOM produit localement, lu par le scanner

2.16 EPIC-015 — PWA déployable et restitution visuelle

Rendre le front servi par la pile, démarrable sans réseau, et capable de montrer les données au lieu de les tabuler.

ID	Story	Pts	Priorité	Critères d'acceptation
US-064	Déploiement de la PWA dans la pile	5	Critique	/ rend la console, plus la console d'administration
US-065	Démarrage hors ligne réel	8	Haute	Précache généré au build ; démarrage à froid sans réseau
US-066	Graphiques des tableaux de bord	8	Haute	Légende et équivalent tabulaire ; jamais la seule couleur
US-067	Fond cartographique réel	8	Haute	Rayons géodésiques ; repli sans accélération matérielle
US-068	Bandeau de mise à jour	3	Moyenne	Mise à jour proposée sans perdre l'écran courant

2.17 EPIC-016 — Fiabilité de la synchronisation et tenue à l'échelle

Qu'une saisie hors ligne ne se duplique ni ne se perde, et que les lectures tiennent sur un parc réel.

ID	Story	Pts	Priorité	Critères d'acceptation
US-055	Idempotence des mutations rejouées	8	Critique	Clé générée avant la première tentative ; 409 si l'empreinte diffère
US-069	Index et garde-fous d'exécution	5	Haute	Index avec le tenant en tête ; délais maximaux posés
US-070	Suppression des N+1 sur les listages	5	Haute	Une requête par listage, vérifié au compteur
US-071	Mesure et plancher de couverture	3	Moyenne	Campagnes fusionnées ; la chaîne échoue sous le plancher

2.18 EPIC-017 — Conformité réglementaire du miel

Étiquetage conforme à la directive (UE) 2024/1438, applicable au 14 juin 2026.

ID	Story	Pts	Priorité	Critères d'acceptation
US-056	Lot de conditionnement et mention d'origine	13	Critique	Parts consolidées par pays, décroissantes, totalisant 100 % ; référence de récolte facultative

2.19 EPIC-018 — Modèle d'autorisation complet

Que le navigateur ne détienne plus de jeton, et qu'un agent ne puisse pas énumérer le parc entier de son exploitation.

ID	Story	Pts	Priorité	Critères d'acceptation
US-072	Traductions en ressources de locale	8	Moyenne	Fichiers ouvrables par un traducteur ; parité tenue à la compilation
US-073	Jetons détenus par le serveur (BFF)	21	Critique	Aucun jeton dans le navigateur ; cookie <code>HttpOnly</code>
US-057	Portée par affectation d'agent	13	Critique	Règle posée dans le SGBD ; l'absence de portée vaut « rien voir »

2.20 EPIC-019 — Dette technique et contrat vérifié

Vider la liste de dettes recensées plutôt que la reconduire de sprint en sprint.

ID	Story	Pts	Priorité	Critères d'acceptation
US-074	Agrégats calculés en base	8	Haute	Prouvés sous le rôle applicatif réel, pas sous le propriétaire
US-075	Scission du service de tableau de bord	3	Moyenne	Contrat HTTP inchangé, aucun test d'intégration retouché
US-076	Parité client / contrat OpenAPI	8	Haute	Une divergence casse la compilation en nommant le champ
US-077	Couche anticorruption vers l'analyse	5	Moyenne	Type d'entrée neutre ; indisponibilité rendue, jamais levée
US-078	Sessions serveur persistées	5	Haute	Schéma créé par les migrations ; la session survit au redéploiement
US-079	Synthèse de pilotage agrégée en base	5	Haute	Poids, motifs et alertes agrégés en SQL ; l'agrégat du tableau de bord est réutilisé
US-080	Courbes journalières côté serveur	8	Haute	Regroupement par intervalle à la demande : l'agrégat continu est refusé sous RLS ; volume divisé par ≈ 95

CHAPITRE 3

Planification des sprints

Le projet est découpé en un **sprint de cadrage** (S0) suivi de **18 sprints de construction de 14 jours**, du 14 juillet 2026 au 26 avril 2027, avec une capacité de référence de 40 points par sprint.

Les sprints 9 à 18 prolongent le plan initial, arrêté à S8. Trois phases s’y distinguent : **construire** (S9 à S11 — exploitation, intelligence spatiale, puis correction de ce qu’un audit du front a révélé), **durcir** (S12 à S14 — sécurité, PWA livrable, conformité miel), et **solder** (S15 à S18 — externalisation des ressources de langue, sessions sans jeton, puis les dettes techniques jusqu’à liste vide).

Le S11 ne suit pas la cadence mécanique de 14 jours, qui l’aurait placé du 15 au 28 décembre, à cheval sur la trêve de fin d’année : il est décalé à janvier plutôt que planifié sur une capacité indisponible. Chaque sprint est décrit ci-dessous avec son *plan d’exécution* : décomposition en tâches techniques, estimation en jours-homme (j-h), livrables et parades aux risques.

Le sprint 0 ne livre aucune fonctionnalité métier et reste hors du calcul de vélocité. Son objet est de lever les quatre décisions structurantes (ADR-001 à ADR-004) et de prouver que la chaîne technique tient de bout en bout *en production* — Spring Boot, PostgreSQL/PostGIS/TimescaleDB, Keycloak, TLS, CI/CD et observabilité assemblés. Sa règle de sortie est stricte : si ce socle n’est pas déployé au terme des deux semaines, le planning est renégocié plutôt qu’absorbé en heures supplémentaires.

Base de chiffrage. Un sprint de 14 jours compte 10 jours ouvrés. Avec une équipe de **trois développeurs à temps plein**, la capacité brute est de $3 \times 10 = 30$ j-h par sprint. Chaque plan d’exécution est chiffré à cette capacité : environ **26,5 j-h** de tâches techniques et **3,5 j-h** de cérémonies Scrum et de revues de code (planning 4 h, dailies, review 2 h, rétrospective 1 h, par personne). Sur les 18 sprints de construction, cela représente **540 j-h** pour 582 points, soit $\approx 0,93$ j-h par story point.

3.1 Calendrier général

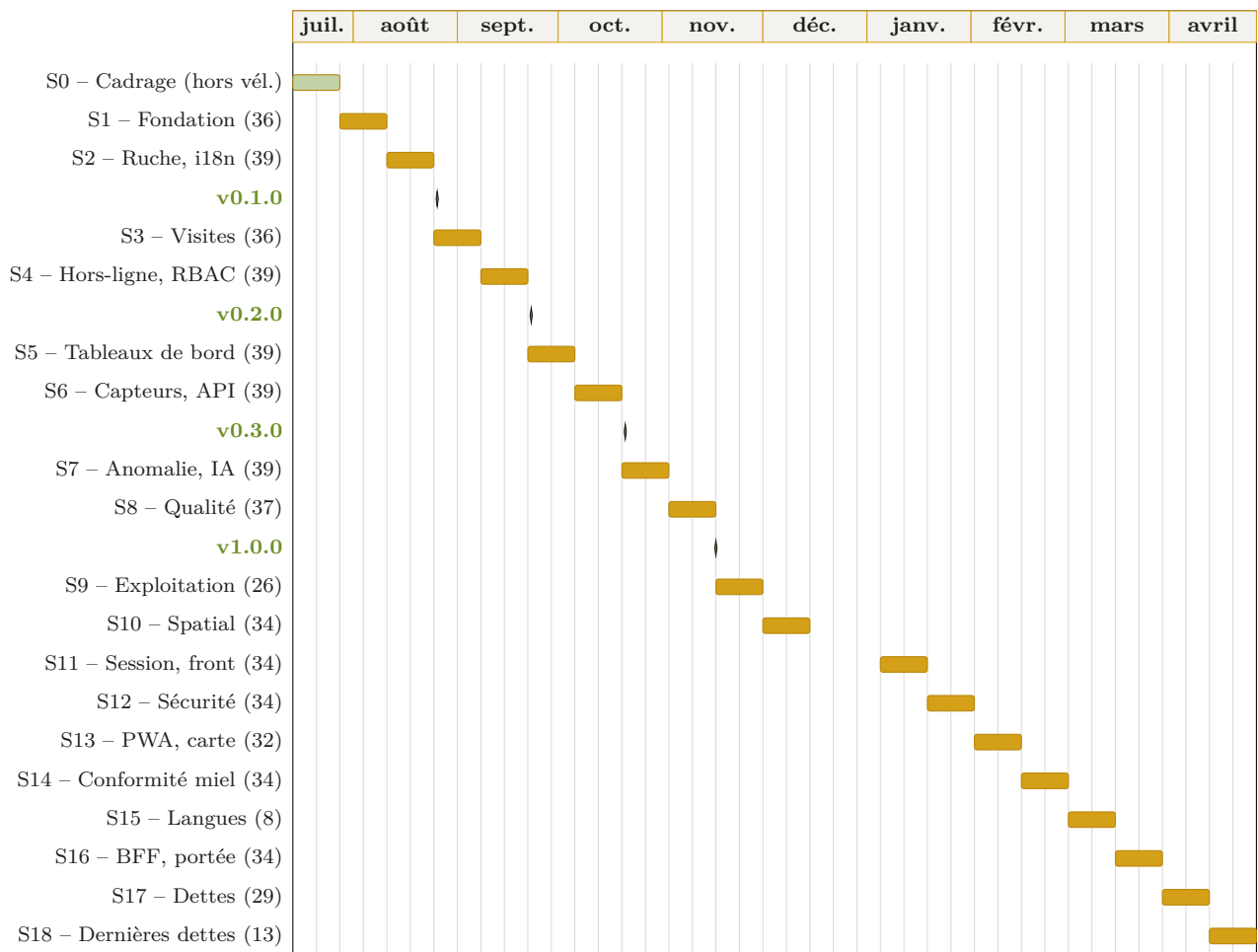


Figure 3.1. Diagramme de Gantt du sprint de cadrage, des 18 sprints de construction et des 4 jalons de release. Le décrochage entre S10 et S11 correspond à la trêve de fin d'année. Les jalons s'arrêtent à v1.0.0 (S8) : les sprints suivants durcissent et soldent un produit déjà livrable, sans nouvelle version majeure.

3.2 Synthèse des sprints

#	Période	Thème	Points ¹	Stories
0	14/07 → 27/07	Cadrage — décisions & walking skeleton	—	ADR-001 à 004
1	28/07 → 10/08	Fondation — CRUD & configuration	36	US-001/002/003/005/006/025
2	11/08 → 24/08	Ruche, i18n & socle qualité	39	US-004/016/023/024/037
3	25/08 → 07/09	Visites & rapports	36	US-007/008/009/010/028
4	08/09 → 21/09	Hors-ligne, auth. & RBAC	39	US-011/019/020/021/022
5	22/09 → 05/10	Tableaux de bord & pilotage	39	US-012/013/014/027/031
6	06/10 → 19/10	Synthèse, capteurs & API	39	US-015/017/018/026/029
7	20/10 → 02/11	Fonctions avancées & IA	39	US-030/032/033/034/038
8	03/11 → 16/11	Qualité & livraison	37	US-035/036/039/040
9	17/11 → 30/11	Exploitation & durcissement	26	US-041/042/043/044
10	01/12 → 14/12	Intelligence spatiale & remise à niveau	34	US-045/046/047/048/049
11	05/01 → 18/01	Fiabilité de la session et du front	34	US-050/051/052/053/054
12	19/01 → 01/02	Durcissement de la sécurité	34	US-058 à 063
13	02/02 → 15/02	PWA livrable, graphiques & carte	32	US-064 à 068
14	16/02 → 01/03	Idempotence, index & conformité miel	34	US-055/056/069/070/071
15	02/03 → 15/03	Ressources de langue externalisées	8	US-072
16	16/03 → 29/03	Sessions sans jeton & portée par affectation	34	US-073/057
17	30/03 → 12/04	Dettes techniques	29	US-074 à 078
18	13/04 → 26/04	Les deux dernières dettes	13	US-079/080
Total			582	80 stories

Charge lissée. La répartition initiale plaçait les sprints 5 (50 pts) et 7 (57 pts) très au-dessus de la vélocité cible, tandis que les sprints 2 et 4 restaient à 26 pts : le projet était chargé à l'envers, la surcharge tombant au moment où arrivent aussi les livrables de soutenance. Après rééquilibrage, tous les sprints tiennent dans une fourchette de **36 à 39 points**, sous la capacité de référence de 40, sans décaler aucune date ni retirer aucune story. Le détail de l'arbitrage figure en section 3.3.

3.3 Recommandations d'équilibrage et de pilotage

1. Points recalculés depuis le backlog, après application du rééquilibrage décrit en section 3.3.

3.3.1 Arbitrage retenu

Quatre leviers étaient envisageables : descoper des stories en *Could have* (A), lisser la charge sur les sprints creux (B), scinder les stories lourdes (C), ou réserver un *buffer* de capacité (D).

Le levier **B** a été retenu comme arbitrage principal : il rééquilibre la charge **sans retirer aucune story du périmètre** ni décaler la moindre date — ce qui n'est pas le cas de l'option A, et ce que l'option D ne permet pas à périmètre constant. Les leviers C et D restent disponibles en cours de route, à décider en sprint planning si la vélocité mesurée s'écarte de l'hypothèse (cf. *Pilotage* ci-dessous).

Story	Mouvement et justification	De	Vers
US-025	Configuration <code>ConfigZumm.ini</code> : les seuils doivent être externalisés avant d'être consommés par les écrans.	S2	S1
US-016	Modèle Mesure : simple entité, sans dépendance à l'ingestion — déplacée pour dégager S6.	S6	S2
US-023	TLS 1.3 / X.509 : livrer le chiffrement au dernier sprint est un contresens de sécurité ; HTTPS doit être actif dès les premiers écrans.	S8	S2
US-037	Tests d'intégration Testcontainers : le harnais doit exister <i>avant</i> d'accumuler 40 stories, pas après.	S8	S2
US-028	Photos d'inspection : quasi-doublon technique d'US-010 (photos de rapport), regroupées dans le même sprint.	S7	S3
US-022	RBAC : le workflow d'approbation superviseur (US-008) suppose déjà des rôles ; remontée au plus près de S3.	S5	S4
US-019	Conversion d'unités (<i>Strategy</i>) : story autonome, sert de variable d'ajustement.	S6	S4
US-031	Liste de tâches et rappels : adhère au calendrier d'US-012.	S7	S5
US-027	Export CSV/TXT : suit les tableaux de bord qu'il exporte.	S6	S5
US-015	Synthèse ROI : décalée d'un sprint pour désengorger S5, sans dépendance descendante.	S5	S6
US-029	Contexte météo local : story de priorité moyenne, sert de variable d'ajustement.	S7	S6
US-038	Tests de charge k6 : mesurer la performance <i>avant</i> le sprint final laisse un sprint pour corriger.	S8	S7

Sprint	S1	S2	S3	S4	S5	S6	S7	S8
Avant	31	26	31	26	50	39	57	44
Après	36	39	36	39	39	39	39	37

L'amplitude passe de **26–57 points** à **36–39 points**. Le sprint 7, qui culminait à 142 % de la capacité, revient à 98 %, et plus aucun sprint n'excède la capacité de référence.

Capacité de référence portée à 40 points. La valeur initiale de 38 points était intenable comme plafond : le backlog de l'époque totalisait exactement $8 \times 38 = 304$ points, ce qui imposait que *chaque* sprint tombe au point près sur 38 — impossible à obtenir avec des estimations Fibonacci (5,

8, 13) sans déformer l'affectation des stories. La capacité de référence est donc fixée à **40 points**, ce qui laisse une marge d'absorption d'environ 5 % et rend la règle vérifiable. Cette valeur reste une hypothèse à recalibrer sur la vélocité mesurée après S2.

3.3.2 Ordre des dépendances

L'ordre de réalisation compte davantage que la charge. Les points suivants ont guidé l'arbitrage ci-dessus :

- **US-022 (RBAC) était trop tardive en S5** : le workflow d'approbation superviseur (US-008, S3) suppose déjà des rôles. Le RBAC minimal reste amorcé en S3 (rôles portés par Agent) et la matrice complète est traitée en S4 ;
- **US-023 (TLS) était en S8** : livrer le chiffrement des échanges au tout dernier sprint expose l'ensemble des environnements intermédiaires. Remontée en S2 ;
- **US-024 (i18n RTL arabe) en S2** : bien placée, mais le rendu RTL doit être testé dès la première maquette — le rétrofit RTL est le piège classique ;
- **Spike technique en début de S1** (2-3 jours) sur PostGIS + Spring Boot : c'est le risque identifié du sprint, autant le neutraliser tôt. De même, prototyper Service Worker/IndexedDB pendant S3 pour dérisquer S4.

3.3.3 Pilotage

- **Recalibrer après S2** : la capacité de 40 pts est une hypothèse ; après deux sprints mesurés, replanifier S5-S8 avec la vélocité réelle (c'est là que le choix entre les options A et B se décide) ;
- **Release interne à chaque sprint** (tag `v0.x.y-sprintN`) même si seules les 4 releases publiques comptent : cela teste le pipeline de release avant la `v1.0.0` ;
- **Calendrier** : si le cadrage ne démarre pas réellement le 14/07, décaler toutes les dates est préférable à un sprint 0 amputé — c'est le sprint dont dépendent toutes les décisions suivantes.

3.3.4 Recommandations techniques transversales

- **Migrations de schéma dès S1** (Flyway ou Liquibase) : chaque évolution du modèle est un script versionné — indispensable avec 4 environnements et des contraintes CHECK qui évoluent ;
- **Décisions d'architecture tracées** (ADR — *Architecture Decision Records*) : un fichier court par choix structurant (JSF vs REST+SPA, stockage des photos, stratégie de synchronisation) — réutilisable tel quel dans le rapport de maintenance ;
- **Stockage des photos** : système de fichiers (ou S3/MinIO) + chemin en base, jamais de BLOB en base — avec limite de taille et génération de miniatures dès l'upload ;
- **Revue de code systématique** : chaque PR relue par un pair avant merge sur `develop` (exigence déjà présente dans la DoD, à outiller par une *branch protection rule*) ;
- **Registre des risques** tenu à jour à chaque rétrospective : les risques par sprint listés ci-dessous en sont l'état initial ;
- **UML au fil de l'eau** : commencer les diagrammes dès S2 et les maintenir à chaque sprint plutôt que de tout produire en S8 (US-039, 13 pts, est la plus grosse story du sprint final) ;
- **Gel des fonctionnalités** (*feature freeze*) au jour 10 du sprint 8 : les 4 derniers jours sont réservés aux corrections, aux répétitions de maintenance et à l'empaquetage des livrables.

3.4 Sprint 1 — Fondation : CRUD & configuration

Objectif	Avoir un socle technique fonctionnel avec les entités principales et la configuration externalisée.
Période	28/07/2026 → 10/08/2026 (14 jours) — 36 pts / capacité 40.
Démo	CRUD Fermier/Ferme/Site/Agent + seuils lus depuis <code>ConfigZumm.ini</code> .
Risques	Complexité PostGIS, configuration Spring Boot / Docker.
Burndown idéal	J1 : 36 — J4 : 27 — J7 : 18 — J10 : 9 — J14 : 0.

ID	Story	Pts
US-001	CRUD Fermier	5
US-002	CRUD Ferme	5
US-003	CRUD Site (avec géolocalisation)	8
US-005	CRUD Agent avec rôles	5
US-006	Contraintes de composition (règles métier)	8
US-025	Configuration <code>ConfigZumm.ini</code>	5

3.4.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Spike PostGIS + Spring Boot : datasource, dialecte spatial, requête lat/long de bout en bout	US-003	3	Note technique + config <code>application.yml</code> versionnée
Squelette Maven (module <code>backend/</code>) + <code>frontend/</code> React + CI GitHub Actions (build + tests)	—	3	Dépôt buildable, pipeline vert
Modèle JPA (Fermier, Ferme, Site, Agent) + migrations Flyway	US-001/002/003/005	3,5	Schéma V1 reproductible
Couche DAO/Service + Bean Validation	US-001/002/003/005	3,5	Services testables
Écrans CRUD des 4 entités	US-001/002/003/005	6	4 CRUD démontrables
Contraintes de composition : CHECK SQL + validateurs Java	US-006	3	Règles métier vérifiées aux deux niveaux
Lecteur <code>ConfigZumm.ini</code> (patron <i>Singleton</i>) + rechargement sans redéploiement	US-025	2	Module de configuration
Tests unitaires des services + jeu de données de démonstration	toutes	2,5	Couverture initiale $\geq 50\%$, seed SQL
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : le spike PostGIS occupe les jours 1–2 ; si l'intégration spatiale bloque, solution de repli : colonnes `lat/long` simples en `NUMERIC` et migration PostGIS différée au sprint 5 (les dashboards n'en dépendent pas avant).

Note d'ordonnancement : US-025 remonte de S2. Externaliser les seuils dès le premier sprint évite

de les coder en dur dans les écrans CRUD puis de les extraire ensuite.

3.5 Sprint 2 — Ruche, i18n & socle qualité

Objectif	Modéliser la hiérarchie ruche/corps/hausse/cadre avec contraintes et outiller la qualité dès le début.
Période	11/08/2026 → 24/08/2026 (14 jours) — 39 pts / capacité 40.
Démo	Composition d'une ruche avec règles métier, bascule FR/EN/AR (RTL) et HTTPS actif.
Risques	Contraintes SQL CHECK, patron <i>Composite</i> , rétrofit RTL.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-004	CRUD Ruche avec composition	13
US-024	Internationalisation (FR/EN/AR)	8
US-016	Modèle de données Mesure	8
US-023	Chiffrement TLS 1.3 / X.509	5
US-037	Tests d'intégration (Testcontainers)	5

3.5.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Modèle <i>Composite</i> ruche/corps/hausse/cadre + contraintes (1 corps, 0-5 hausses, 1-10 cadres/élément)	US-004	4	Migration V2 + entités JPA
Interface de composition (vue arborescente, ajout/retrait d'éléments avec validation immédiate)	US-004	4	Écran composition démontrable
i18n FR/EN/AR : externalisation des libellés en ressources de locale, bascule de langue, gabarit RTL	US-024	4	Application trilingue
Test de rendu RTL sur les écrans existants (S1 + composition)	US-024	1,5	Rapport d'écarts RTL
Entité Mesure (horodatage, géolocalisation, unité, source) + migration	US-016	2,5	Modèle extensible capteurs
TLS 1.3 : certificats X.509, HTTPS forcé, redirection 80 → 443 sur staging	US-023	3	Chaîne TLS validée dès S2
Socle de tests d'intégration Testcontainers (PostgreSQL réel, PostGIS + TimescaleDB)	US-037	4	Premier test d'intégration vert en CI
Déploiement continu staging (docker-compose)	—	3,5	Environnement staging alimenté à chaque merge
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : implémenter les règles de composition d'abord en Java (testable rapidement),

puis répliquer en CHECK SQL ; si le patron *Composite* complique le mapping JPA, se limiter à une hiérarchie à trois tables avec clés étrangères et compteurs contraints.

Note d'ordonnement : ce sprint absorbe US-016, US-023 et US-037. Le harnais Testcontainers et la chaîne TLS sont des prérequis d'infrastructure : les placer en S8, comme le faisait la répartition initiale, revenait à valider tardivement des choix qui conditionnent tous les sprints intermédiaires. US-016 n'est qu'une entité et ne dépend pas de l'ingestion (US-017/018, S6).

3.6 Sprint 3 — Visites & rapports

Objectif	Workflow complet : planification → approbation → réalisation → rapport.
Période	25/08/2026 → 07/09/2026 (14 jours) — 36 pts / capacité 40.
Démo	Workflow de visite avec rapport complet et photos d'inspection.
Risques	Volumétrie des photos ; mode hors-ligne (reporté au sprint 4).
Burndown idéal	J1 : 36 — J4 : 27 — J7 : 18 — J10 : 9 — J14 : 0.

ID	Story	Pts
US-007	Planifier une visite	8
US-008	Approuver/refuser un planning	5
US-009	Réaliser une visite et remplir le rapport	13
US-010	Ajouter des photos au rapport	5
US-028	Photos d'inspection	5

3.6.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Modèle Visite/Planning/Rapport + machine à états (planifiée → approuvée → réalisée / refusée)	US-007/008/009	3,5	Migration V3 + diagramme d'états
RBAC minimal : rôles portés par Agent + garde d'accès sur le workflow (anticipation d'US-022, cf. §3.3)	US-008	2,5	Approbation réservée au superviseur
Écrans de planification (date, raison, actions prévues)	US-007	3,5	Formulaire de planification
File d'approbation du superviseur (accepter/refuser + motif)	US-008	2,5	Écran d'approbation
Formulaire de rapport : date, heure, durée, constatations, évaluations 1–3	US-009	4,5	Rapport complet saisissable
Upload multi-photos : stockage fichiers + miniatures + limite de taille	US-010	3,5	Photos attachées au rapport
Réutilisation du module photo pour l'inspection (même pipeline d'upload, cible différente)	US-028	1,5	Photos liées à l'inspection
Spike Service Worker + IndexedDB (préparation S4)	—	2	Prototype de mise en cache
Tests unitaires du workflow (transitions interdites)	US-007/008/009	3	Machine à états couverte
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : le report du mode hors-ligne est déjà acté (S4) ; le spike Service Worker de fin de sprint sécurise cette échéance. Verrouiller la machine à états par des tests avant de brancher l'interface. Stockage des photos sur système de fichiers avec chemin en base et limite de taille — jamais de BLOB (cf. §3.3.4).

Note d'ordonnancement : US-028 remonte de S7. C'est techniquement le même module d'upload qu'US-010 ; les traiter dans le même sprint évite de rouvrir le sujet quatre sprints plus tard pour 1 j-h de travail.

3.7 Sprint 4 — Hors-ligne, authentification & RBAC

Objectif	Permettre la saisie terrain sans réseau et verrouiller les accès par rôle.
Période	08/09/2026 → 21/09/2026 (14 jours) — 39 pts / capacité 40.
Démo	PWA hors-ligne, OAuth Google + repli local, matrice RBAC appliquée.
Risques	Service Workers, IndexedDB, synchronisation différée.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-011	Mode hors-ligne et synchronisation différée	13
US-020	Authentification OAuth 2.0 Google	8
US-021	Authentification locale (repli)	5
US-022	Contrôle d'accès RBAC	8
US-019	Conversion d'unités hétérogènes	5

3.7.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
PWA : manifest + Service Worker (cache de l'app shell)	US-011	3,5	Application installable
File d'attente IndexedDB des saisies hors-ligne (visites, rapports)	US-011	4	Saisie sans réseau fonctionnelle
Synchronisation différée : rejeu à la reconnexion, idempotence, résolution de conflits (<i>last-write-wins</i> + journal)	US-011	4,5	Démo « mode avion »
OAuth 2.0 Google : flot <i>Authorization Code</i> + émission JWT	US-020	3,5	Connexion Google opérationnelle
Authentification locale de repli (hachage BCrypt/Argon2)	US-021	2	Repli fonctionnel hors OAuth
Gestion des secrets dans le pipeline (GitHub Secrets)	US-020	1	Aucun secret en clair dans le dépôt
Généralisation RBAC : matrice 6 profils × 15 fonctions (extension du RBAC minimal de S3)	US-022	3	Matrice appliquée à tous les écrans
Conversion d'unités (patron <i>Strategy</i>)	US-019	2	Conversions couvertes par tests
Tests E2E du parcours hors-ligne (Cypress, réseau coupé)	US-011	3	Scénario automatisé en CI
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : limiter le périmètre hors-ligne à la *saisie de visite/rapport* (pas de consultation complète) ; en cas de blocage sur la résolution de conflits, dégrader en « premier arrivé, premier servi » avec signalement manuel des collisions.

Note d'ordonnancement : US-022 remonte de S5. Le RBAC complet doit suivre immédiatement le RBAC minimal amorcé en S3 et précéder les tableaux de bord, dont la visibilité dépend du profil. US-019 (*Strategy*), story autonome sans dépendance, sert de variable d'ajustement : c'est le fusible du sprint si le hors-ligne dérive.

3.8 Sprint 5 — Tableaux de bord & pilotage

Objectif	Donner à l'apiculteur ses trois vues de pilotage quotidien.
Période	22/09/2026 → 05/10/2026 (14 jours) — 39 pts / capacité 40.
Démo	Calendrier matriciel, tableaux production et alertes, export CSV.
Risques	Performance des agrégations SQL, Chart.js.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-012	Calendrier matrice agents × ruches	13
US-013	Tableau de bord production	8
US-014	Tableau de bord alertes sanitaires	8
US-031	Liste de tâches et rappels	5
US-027	Export CSV/TXT	5

3.8.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Vues SQL agrégées + index (production, visites par période)	US-012/013	4	Requêtes < 200 ms sur données de test
Calendrier matrice agents × ruches : affichage + pop-up de détail	US-012	4,5	Matrice navigable
Filtres mois/semaine/saison/année du calendrier	US-012	2,5	Filtres opérationnels
Dashboard production (Chart.js) + seuils paramétrables via <code>ConfigZümm.ini</code>	US-013	4,5	Graphiques de production
Dashboard alertes sanitaires : règle de baisse anormale (seuil simple, l'EWMA arrive en S7)	US-014	4,5	Liste d'alertes + statut inspection
Liste de tâches et rappels adossée au calendrier	US-031	3	Rappels démontrables
Export CSV/TXT des données maîtrisées par l'utilisateur	US-027	3,5	Export téléchargeable
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : matérialiser les agrégations coûteuses (vues matérialisées rafraîchies à la demande) plutôt que d'optimiser des requêtes à chaud. Si le burndown dérive, scinder US-012 en « affichage matrice » (8 pts) et « filtres/pop-up » (5 pts) — levier C — plutôt que de descoper : les filtres sont la partie la moins critique pour la démo.

Note d'ordonnancement : US-015 (ROI, 13 pts) descend en S6, ce qui ramène ce sprint de 50 à 39 points. US-031 et US-027 remontent car ils s'adossent respectivement au calendrier et aux tableaux qu'ils exportent.

3.9 Sprint 6 — Synthèse, capteurs & API

Objectif	Compléter les tableaux de bord par la synthèse ROI et ouvrir le système aux capteurs et aux tiers.
Période	06/10/2026 → 19/10/2026 (14 jours) — 39 pts / capacité 40.
Démo	Synthèse ROI, ingestion REST/MQTT d'une mesure, appel REST depuis un client externe généré depuis OpenAPI.
Risques	MQTT, idempotence, quota d'appels météo.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-015	Tableau de bord synthèse et ROI	13
US-017	Interface ingestion REST/MQTT	8
US-018	Seuils paramétrables et logique d'alerte	8
US-026	Service web REST <code>getZummHoneyActualQuantity</code>	5
US-029	Contexte météo local	5

3.9.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Synthèse ROI : agrégation production, interventions, coûts + graphiques	US-015	5	Page de synthèse
API REST d'ingestion <code>POST /api/mesures</code> idempotente (clé d'unicité source + horodatage)	US-017	4	Endpoint documenté (OpenAPI)
Broker MQTT (Mosquitto en docker-compose) + consommateur	US-017	4	Ingestion par topic démontrable
Seuils paramétrables + hystérésis/anti-rebond	US-018	3,5	Alertes sans battement
Endpoint REST <code>getZummHoneyActualQuantity</code> + contrat OpenAPI 3	US-026	3,5	Client tiers généré depuis le contrat
API météo par géolocalisation + cache local (quota d'appels)	US-029	3	Contexte météo sur la visite
Tests de contrat des API (REST, validation OpenAPI)	US-017/026	3,5	Suites de contrat en CI
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : si MQTT dérape, livrer d'abord la voie REST (même modèle de données) et brancher MQTT en simple adaptateur ; l'idempotence est testée avant tout branchement de source réelle. US-029 (priorité moyenne, aucune dépendance descendante) est le fusible du sprint.

Note d'ordonnancement : l'entité Mesure (US-016) a été livrée en S2, ce sprint n'a donc plus qu'à brancher l'ingestion dessus. US-015 descend de S5, US-019 remonte en S4.

3.10 Sprint 7 — Fonctions avancées & détection d'anomalie

Objectif	Livrer les fonctions différenciantes et la détection d'anomalie sur les mesures.
Période	20/10/2026 → 02/11/2026 (14 jours) — 39 pts / capacité 40.
Démo	Carte et rayons de butinage, suivi de reine, QR code de lot, alerte EWMA.
Risques	Calibrage de la ligne de base EWMA sans historique réel.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-034	Détection d'anomalie adaptative (EWMA)	13
US-030	Carte et rayons de butinage	8
US-032	Suivi de la reine	5
US-033	Récolte et QR code	8
US-038	Tests de charge k6	5

3.10.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Algorithme EWMA : ligne de base par ruche + z-score, jeu de données simulé pour calibrage	US-034	5,5	Détection validée sur données de test
Intégration EWMA → alertes sanitaires (remplace le seuil simple de S5)	US-034	3	Alerte adaptative en démo
Carte MapLibre GL + OpenStreetMap : sites + cercles de butinage 1/2/3 km	US-030	5	Carte interactive
Suivi de la reine : historique de statut	US-032	3	Chronologie reine
Récolte : lot + QR code + page publique de traçabilité	US-033	5	QR scannable de bout en bout
Tests de charge k6 sur les parcours critiques + corrections de performance	US-038	5	p95 < 500 ms, erreurs < 1 %
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : l'EWMA est calibrée sur un jeu de données simulé — sans historique réel, la ligne de base n'a pas de sens statistique, ce point doit être assumé explicitement en soutenance. Si le calibrage dérape, l'application reste sur le seuil simple de S5 sans autre impact. US-033 est le fusible du sprint (priorité moyenne, aucune dépendance descendante).

Note d'ordonnement : US-038 remonte de S8. Mesurer la performance à l'avant-dernier sprint laisse un sprint entier pour corriger ce qui est trouvé — la placer en S8 revenait à découvrir les problèmes sans avoir le temps de les traiter. US-035 (interface microservice IA) descend en S8 : l'algorithme est ici, son extraction en service découplé l'est là.

3.11 Sprint 8 — Qualité & livraison

Objectif	Atteindre les critères de la DoD sur l'ensemble du produit et préparer les livrables de soutenance.
Période	03/11/2026 → 16/11/2026 (14 jours) — 37 pts / capacité 40.
Démo	Soutenance blanche : démonstration complète, rapport, poster.
Risques	Couverture de tests, concentration documentaire.
Burndown idéal	J1 : 37 — J4 : 28 — J7 : 19 — J10 : 9 — J14 : 0.

ID	Story	Pts
US-035	Interface microservice IA Python	8
US-036	Tests unitaires JUnit 5 + Mockito	8
US-039	Diagrammes UML complets	13
US-040	Rapport + poster + présentation	8

3.11.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Extraction de l'EWMA en microservice Python découplé (API REST/JSON, conteneur docker-compose)	US-035	5	Conteneur IA déployé
Campagne de tests unitaires : combler les trous jusqu'à 70 % de couverture métier	US-036	7	Rapport JaCoCo $\geq 70\%$
Finalisation des 10 diagrammes UML (maintenus depuis S2, cf. §3.3.4)	US-039	6	Dossier UML complet
Rapport, poster, présentation + répétition de soutenance	US-040	8,5	Livrables de soutenance
<i>Feature freeze</i> au jour 10 : corrections et empaquetage uniquement	—	—	v1.0.0 taguée et déployée
Cérémonies Scrum (planning, dailies, review, rétrospective) et revues de code	—	3,5	Rituels tenus, PR relues avant merge
Total		30	Capacité du sprint

Parades aux risques : la couverture de tests ne se rattrape pas en un sprint — d'où le quality gate progressif (chapitre 5) qui doit amener la couverture à $\approx 65\%$ dès la fin de S7 ; les diagrammes UML maintenus au fil de l'eau réduisent US-039 à une passe de mise en cohérence.

Note d'ordonnancement : ce sprint perd US-023 (TLS, → S2), US-037 (Testcontainers, → S2) et US-038 (k6, → S7). Le sprint final ne conserve ainsi que ce qui ne peut pas être anticipé — la campagne de couverture, la mise en cohérence documentaire et la soutenance — au lieu de cumuler infrastructure de sécurité, harnais de test et documentation dans les deux dernières semaines.

3.12 Sprint 9 — Exploitation & durcissement

Objectif	Compléter le produit par des fonctionnalités d'exploitation à valeur immédiate et fiabiliser le déploiement de production.
Période	17/11/2026 → 30/11/2026 (14 jours) — 26 pts / capacité 40.
Démo	Notification d'alerte, export PDF d'un rapport de visite, journal d'audit et widget de prévision de récolte.
Risques	Dépendances tierces (SMTP, bibliothèque PDF) ; durcissement de production — isolation de la base Keycloak, validation de l'émetteur des jetons.
Burndown idéal	J1 : 26 — J4 : 20 — J7 : 13 — J10 : 7 — J14 : 0.

ID	Story	Pts
US-041	Notifications e-mail des alertes de seuil	5
US-042	Prévision de récolte (tendance du poids)	8
US-043	Journal d'audit	8
US-044	Rapport de visite au format PDF	5

Parades aux risques : les notifications sont désactivées par défaut — aucune dépendance SMTP en local ni en intégration continue, l'activation se fait par configuration en production. L'isolation de la base Keycloak n'est effective que sur volume vierge : la procédure de reprise l'indique.

3.13 Sprint 10 — Intelligence spatiale & remise à niveau

Objectif	Transformer la carte descriptive en outil d'aide à la décision terrain, et remettre la roadmap publiée en phase avec le produit réellement livré.
Période	01/12/2026 → 14/12/2026 (14 jours) — 34 pts / capacité 40.
Démo	Grappes de sites sur la carte, voisins d'un site, tournée ordonnée d'un agent ; roadmap recompilée et CI front (lint + tests).
Risques	Rendu des grappes sur la carte SVG autonome (MapLibre non intégré depuis S7) ; attente d'un routage routier réel sur US-047.
Burndown idéal	J1 : 34 — J4 : 26 — J7 : 18 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-045	Regroupement spatial des sites (clustering)	8
US-046	Distances inter-sites et plus proches voisins	5
US-047	Ordre de tournée optimisé	8
US-048	Alignement du backlog produit et de la roadmap	5
US-049	Socle de tests et de linting du front-end	8

Parades aux risques : le regroupement et l'ordonnancement sont calculés côté serveur et couverts par des tests d'intégration sur un PostGIS réel — ils restent démontrables indépendamment du rendu cartographique. Le caractère *indicatif* de l'ordre de tournée (distances à vol d'oiseau, heuristique non

optimale) est affiché dans l'interface et documenté dans le contrat OpenAPI, pour ne pas laisser croire à un calcul d'itinéraire routier.

3.14 Sprint 11 — Fiabilité de la session et du front

Objectif	Rendre la console utilisable une journée entière sur le terrain sans déconnexion ni perte de saisie, et lever les approximations d'interface héritées du prototypage.
Période	05/01/2027 → 18/01/2027 (14 jours) — 34 pts / capacité 40.
Démo	Session tenue plus de trente minutes, navigation au bouton retour et partage d'URL, liste paginée, bascule FR → AR des dates et des nombres, suppression confirmée au clavier seul.
Risques	Le rafraîchissement du jeton touche le chemin critique de toutes les requêtes ; première dépendance de routage à trancher par ADR ; offline_access à vérifier sur le realm dès le jour 1.
Burndown idéal	J1 : 34 — J4 : 26 — J7 : 18 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-050	Rafraîchissement du jeton OIDC	8
US-051	Navigation adressable par URL	8
US-052	Pagination des listes	8
US-053	Formatage localisé des dates et des nombres	5
US-054	Dialogues du <i>design system</i>	5

Origine. Ce sprint ne vient pas du cahier des charges mais d'un audit du front mené après le SPRINT-10. Le défaut qui le motive : le flux OIDC ne conserve pas le *refresh token*, et le realm laisse la durée de vie du jeton d'accès à son défaut de cinq minutes — l'utilisateur est donc déconnecté toutes les cinq minutes, en perdant sa saisie. Rien ne l'avait révélé : le flux OIDC n'est joué ni en intégration continue, ni en test.

Parades aux risques : la logique de rafraîchissement est isolée dans un module testé unitairement *avant* d'être branchée dans le client d'API — elle traverse sinon toutes les requêtes de l'application. US-050 et US-051 sont menées ensemble : rétablir la session sans rendre les écrans adressables ramènerait l'utilisateur sur le premier onglet à chaque reconnexion.

3.15 Sprint 12 — Durcissement de la sécurité

Objectif	Qu'aucune garantie annoncée au cahier ne repose sur une configuration absente, un contrôle manquant ou une règle jamais appliquée.
Période	19/01/2027 → 01/02/2027 (14 jours) — 34 pts / capacité 40.
Démo	Six tentatives d'intrusion jouées devant les parties prenantes, sur l'image d'avant puis celle du jour : jeton sans <code>tenant_id</code> (403), jeton du client <code>account</code> (401), position d'un site lue par un non-propriétaire (arrondie), suppression par un compte sans rôle métier (403), script <code>inline</code> (bloqué par la CSP), dépendance vulnérable poussée (chaîne rouge).
Risques	Le refus par défaut sur <i>tous</i> les endpoints coupe une fonctionnalité entière si un rôle est oublié ; le mapper <code>tenant_id</code> doit exister dans les <i>deux</i> realms ; la CSP peut casser la PWA sans échec côté serveur ; rendre une porte d'intégration continue bloquante sans maîtriser sa source de données.
Burndown idéal	J1 : 34 — J3 : 29 — J6 : 22 — J9 : 14 — J11 : 8 — J13 : 3 — J14 : 0.

ID	Story	Pts
US-058	Claim <code>tenant_id</code> garanti et refus explicite	5
US-059	Validation d'audience du jeton	3
US-060	Confidentialité des positions de ruchers	8
US-061	RBAC en refus par défaut et identité machine	8
US-062	En-têtes de sécurité du proxy inverse	5
US-063	Portes de sécurité bloquantes dans la CI	5

Origine. Une revue d'architecture menée sur la branche principale a relevé six écarts entre ce que la documentation garantissait et ce que le code faisait. Aucun n'était visible en test : tous se manifestaient à l'exécution, en configuration de production.

Ce que la revue a rapporté, et qui n'était pas au programme. La démonstration de la dernière porte, jouée sur l'arbre de dépendances réel, a rendu **29 vulnérabilités, dont trois de score 9.1 à 9.6** — Spring Boot figé depuis dix-huit mois. La leçon consignée : aucune mesure de configuration ne compense une dépendance non tenue à jour, et l'absence de garde opérant l'avait laissé invisible.

Parades aux risques : chaque correctif porte un test de régression qui échoue si on le retire — un réglage de sécurité sans test n'est pas une garantie, c'est une intention. Une porte d'intégration continue n'est rendue bloquante que si son entrée est produite *localement* (SBOM), jamais si elle dépend d'un registre tiers interrogé à chaud.

3.16 Sprint 13 — PWA livrable, graphiques et carte

Objectif	Que la PWA soit servie par la pile, démarre sans réseau, et montre les données au lieu de les tabuler.
Période	02/02/2027 → 15/02/2027 (14 jours) — 32 pts / capacité 40.
Démo	Revue jouée entièrement depuis la pile déployée : installation de la PWA, démarrage à froid Wi-Fi coupé, graphiques doublés de leur équivalent tabulaire, carte à rayons géodésiques comparée entre Bizerte et la Norvège, bandeau de mise à jour déclenché en séance, bascule clair/sombre puis FR → AR en RTL.
Risques	Un <i>service worker</i> mal réglé sert indéfiniment une version périmée ; le fond cartographique tiers expose les positions des ruchers ; MapLibre alourdit le paquet initial ; graphiques maison et accessibilité ; licence AGPL introduite par une dépendance.
Burndown idéal	J1 : 32 — J3 : 27 — J6 : 20 — J9 : 13 — J12 : 5 — J14 : 0.

ID	Story	Pts
US-064	Déploiement de la PWA dans la pile	5
US-065	Démarrage hors ligne réel (précache généré)	8
US-066	Graphiques des tableaux de bord	8
US-067	Fond cartographique réel (MapLibre + OSM)	8
US-068	Bandeau de mise à jour de la PWA	3

Origine. « Ça marche en développement » ne dit rien du déploiement : le front était complet et testé depuis trois sprints, et n'était servi par personne.

Ce que la revue a rapporté. La palette catégorielle de la charte, validée *par calcul* plutôt qu'à l'œil, s'est révélée avoir deux emplacements adjacents en deçà du plancher de distinction (ΔE OKLab de 12,9 pour un plancher de 15). Le défaut était rigoureusement invisible en relecture.

Parades aux risques : no-cache sur la coquille et cache immuable réservé aux seuls fichiers dont le nom porte une empreinte ; tuiles cartographiques paramétrables, seule option qui garantisse qu'aucune position de rucher ne sorte du système ; MapLibre chargé paresseusement dans son propre morceau (78 ko initiaux, 252 ko à l'ouverture de la carte seulement).

3.17 Sprint 14 — Idempotence, index et conformité miel

Objectif	Qu'une saisie hors ligne ne se duplique ni ne se perde, que les lectures tiennent sur un parc réel, et que l'étiquette soit conforme au 14 juin 2026.
Période	16/02/2027 → 01/03/2027 (14 jours) — 34 pts / capacité 40.
Démo	Revue jouée sur le scénario de terrain : coupure entre requête et réponse (une seule visite créée), jeton expiré pendant la coupure (saisie conservée au lieu d'être détruite), même clé et corps différent (409), lot mêlant deux récoltes françaises, une tunisienne et un miel acquis à un tiers, plan d'exécution avant et après index, compteur de requêtes sur les six listages, couverture mesurée à 82,6 %.
Risques	Une clé d'idempotence régénérée au rejeu ne protège de rien ; mémoriser les réponses en échec rendrait une erreur transitoire définitive ; rendre la récolte obligatoire dans un lot rendrait la mention légale fausse ; un plancher de couverture posé sur la seule campagne unitaire serait trompeur.
Burndown idéal	J1 : 34 — J4 : 26 — J7 : 18 — J10 : 11 — J12 : 6 — J14 : 0.

ID	Story	Pts
US-055	Idempotence des mutations rejouées	8
US-056	Lot de conditionnement et mention d'origine	13
US-069	Index et garde-fous d'exécution sur les mesures	5
US-070	Suppression des N+1 sur les listages	5
US-071	Mesure et plancher de couverture	3

Origine. Partir du scénario de terrain — « le réseau tombe entre la requête et la réponse » — plutôt que de la fonctionnalité. C'est ce cadrage qui a fait apparaître deux défauts du rejeu, dont un de *perte de données* que personne n'aurait signalé : l'utilisateur aurait conclu qu'il avait oublié de saisir.

Ce que la revue a rapporté. La démonstration de la compression temporelle, pourtant au programme, **a échoué en séance** : le SGBD refuse la compression sur une table soumise à la sécurité au niveau ligne. Les décisions ADR-001 et ADR-002 étaient incompatibles depuis leur rédaction — personne n'avait tenté de les appliquer ensemble. Arbitrage rendu devant les parties prenantes et consigné en ADR-008.

Parades aux risques : la clé d'idempotence est générée *avant* la première tentative et conservée jusqu'au rejeu ; seules les réponses réussies sont mémorisées ; la référence à la récolte reste facultative, faute de quoi le miel acquis à un tiers deviendrait inreprésentable et les pourcentages ne totaliseraient jamais 100 %.

3.18 Sprint 15 — Ressources de langue externalisées

Objectif	Que les traductions soient des ressources qu'un traducteur peut ouvrir, et qu'une session ne télécharge que la langue qu'elle affiche.
Période	02/03/2027 → 15/03/2027 (14 jours) — 8 pts / capacité 40.
Démo	Une clé est <i>réellement</i> supprimée d'une traduction en séance : la compilation échoue en nommant la clé et la langue. Clé en trop et valeur vide attrapées par le test. Paquet initial 243 → 228 ko. Bascule FR → AR sur réseau ralenti : direction du document immédiate, libellés différés, garde d'annulation sur deux bascules rapprochées.
Risques	Perdre la garantie de parité en externalisant les chaînes ; réécrire 315 clés à la main ; écran vide pendant le chargement ; sprint court laissant croire à une capacité disponible.
Burndown idéal	J1 : 8 — J3 : 6 — J6 : 4 — J9 : 2 — J12 : 1 — J14 : 0.

ID	Story	Pts
US-072	Traductions en ressources de locale, chargées à la demande	8

Note de planification. Sprint volontairement court : il solde un point ouvert plutôt qu'il n'ouvre un chantier, et sa capacité restante est fléchée vers le sprint suivant.

Parades aux risques : la garantie de parité n'est pas perdue mais *reportée* sur le type de retour des chargeurs, et prouvée en la cassant volontairement ; les 315 clés sont extraites par un script exécuté depuis le module lui-même, jamais recopiées — une conversion manuelle de mille lignes d'apostrophes typographiques et de texte arabe est une source de fautes silencieuses.

3.19 Sprint 16 — Sessions sans jeton et portée par affectation

Objectif	Que le navigateur ne détienne plus de jeton, et qu'un agent ne puisse plus énumérer le parc entier de son exploitation.
Période	16/03/2027 → 29/03/2027 (14 jours) — 34 pts / capacité 40.
Démo	Stockage local ouvert en séance : plus aucun jeton. Extrait hostile exécuté dans la console : rien à exfiltrer. Compte apiculteur affecté à trois ruches sur un parc de plus de deux cents : trois lignes rendues. Requête émise sans portée posée : rien. Changement d'adresse électronique : l'affectation tient.
Risques	Le BFF devient un point de passage unique de toutes les requêtes ; session serveur et réplication horizontale ; poser la portée dans les services plutôt qu'en base ; poser le tenant et la portée dans deux requêtes distinctes ; lier le compte à l'agent par le courriel.
Burndown idéal	J1 : 34 — J4 : 26 — J7 : 18 — J9 : 13 — J11 : 8 — J14 : 0.

ID	Story	Pts
US-073	BFF : jetons détenus par le serveur	21
US-057	Portée d'autorisation par affectation d'agent	13

Origine. L'isolation posée en ADR-001 séparait les exploitations entre elles, mais à *l'intérieur* d'une exploitation, tout le monde voyait tout : un saisonnier, un stagiaire ou un compte compromis pouvait énumérer l'intégralité du parc — donc la carte des ruchers. C'est aussi le sprint qui solde la dette du stockage des jetons, ouverte en ADR-006 et déclarée bloquante pour la mise en production au sprint 12.

Ce que la revue a rapporté. La première version des tests était *verte à tort* : en test, l'application se connecte avec le propriétaire de la base, lequel ignore la sécurité au niveau ligne. Les tests ont été refaits sous le rôle applicatif. Règle posée en séance : un test vert ne dit rien tant qu'on n'a pas vérifié qu'il peut rougir.

Parades aux risques : la règle de portée est posée *dans le SGBD*, comme l'isolation de tenant — la poser dans les services reviendrait à ajouter une condition à chaque requête et à espérer que personne ne l'oublie. Le tenant et la portée sont posés dans la *même* requête préparée, faute de quoi une fenêtre s'ouvrirait pendant laquelle une requête verrait toute l'exploitation. L'absence de portée vaut « rien voir », jamais « tout voir ».

3.20 Sprint 17 — Dettes techniques

Objectif	Que les tableaux de bord tiennent sur un parc réel, que le contrat publié soit vérifié, et que la session serveur survive à un redéploiement.
Période	30/03/2027 → 12/04/2027 (14 jours) — 29 pts / capacité 40.
Démo	Parc de 500 ruches et huit millions de relevés : saturation mémoire avant, page rendue après, plan d'exécution projeté. Contrat HTTP identique après scission en trois services. Champ renommé côté serveur : la compilation du client échoue en le nommant. Microservice d'analyse arrêté en séance : la consultation continue, servie par le repli local. Redémarrage du back-end en direct : la session survit, un second nœud est démarré derrière le proxy.
Risques	Descendre les agrégats en SQL contourne la sécurité au niveau ligne si la requête est mal écrite ; scinder un service exposé peut changer le contrat ; un port mal typé laisse fuir le modèle ; deux artefacts dérivés de plus à garder frais ; exception motivée à la règle « toute table porte son tenant et sa politique ».
Burndown idéal	J1 : 29 — J3 : 24 — J5 : 20 — J8 : 13 — J10 : 9 — J12 : 4 — J14 : 0.

ID	Story	Pts
US-074	Agrégats des tableaux de bord calculés en base	8
US-075	Scission du service de tableau de bord	3
US-076	Parité garantie entre client TypeScript et contrat OpenAPI	8
US-077	Couche anticorruption vers le microservice d'analyse	5
US-078	Sessions serveur persistées en base	5

Origine. Ce sprint ne livre aucune fonctionnalité nouvelle. Trois de ses points figuraient *mot pour mot* dans la section « ce qui reste ouvert » des sprints 14, 15 et 16. Une dette reconduite trois fois de suite n'est plus une dette : c'est une décision implicite de ne jamais la payer.

Ce que la revue a rapporté. Le contrôle de parité a trouvé un défaut réel dès sa première exécution, et un défaut dans le contrat *publié* : une heure y était décrite comme un objet structuré alors que l'API sérialise une chaîne. Tout intégrateur tiers générant son client depuis ce contrat aurait produit du code cassé.

Parades aux risques : les requêtes d'agrégation sont jouées sous le rôle applicatif réel, jamais sous le propriétaire de la base — application directe de la leçon du sprint précédent. La réponse HTTP est comparée avant et après la scission : une scission qui change le contrat n'est pas une scission. Les deux artefacts dérivés versionnés sont soumis au même contrôle de fraîcheur que les PDF du cahier des charges.

3.21 Sprint 18 — Les deux dernières dettes

Objectif	Que la synthèse cesse d'être la page la plus coûteuse de l'application, et que les courbes ne transportent plus cent fois trop de points.
Période	13/04/2027 → 26/04/2027 (14 jours) — 13 pts / capacité 40.
Démo	Écran d'accueil du responsable chronométré avant et après. Échec de US-080 montré tel quel en séance, commande à l'appui. Bénéfice obtenu autrement : trois ans d'historique d'une ruche passent de $\approx 105\,000$ points transportés à $\approx 1\,100$, les deux courbes projetées côte à côte étant indiscernables. Série brute conservée pour la détection d'anomalie.
Risques	Une user story dont la faisabilité n'a pas été vérifiée au planning ; répéter un défaut déjà corrigé ailleurs ; une agrégation à la demande qui remplace un cache ; une requête native qui échappe au discriminant de tenant et ne garde qu'une barrière sur deux.
Burndown idéal	J1 : 13 — J4 : 9 — J7 : 6 — J9 : 4 — J11 : 2 — J14 : 0.

ID	Story	Pts
US-079	Synthèse de pilotage agrégée en base	5
US-080	Courbes journalières agrégées côté serveur	8

Origine. La synthèse portait le défaut déjà corrigé ailleurs : elle chargeait toutes les mesures de poids

du tenant pour n'en retenir qu'une valeur par ruche — exactement ce que le sprint précédent avait corrigé sur le tableau de bord, mais laissé ici. Autrement dit, l'écran d'accueil du responsable était la page la plus coûteuse de l'application.

Ce que la revue a rapporté. US-080 n'a pas été livrée comme elle avait été planifiée. L'agrégat continu envisagé est refusé par le SGBD sur une table soumise à la sécurité au niveau ligne — seconde manifestation du conflit tranché en ADR-008 après la compression — et le contournement envisagé n'a même pas pu être tenté, le refus tombant à la *création* et non à la lecture. L'ADR-008 a donc été **généralisé** plutôt que complété au cas par cas : sous sécurité au niveau ligne, aucune fonctionnalité *matérialisante* de l'extension temporelle n'est disponible ; restent le partitionnement, les index, le regroupement par intervalle et les agrégations de dernière valeur — c'est-à-dire l'essentiel. Écrire la règle une fois évite de la redécouvrir une troisième fois.

Parades aux risques : le défaut a été cherché par son *motif* (chargement complet suivi d'une réduction) et non par son symptôme, sur tout le paquet des services. Les tests d'intégration ont attrapé deux régressions que les tests unitaires laissaient passer, dont une sérieuse : une requête *native* échappe au discriminant de tenant, qui ne réécrit que les requêtes de haut niveau. En production la sécurité au niveau ligne couvre ce trou, mais le projet a toujours voulu *deux* barrières — les trois requêtes natives portent désormais un filtre de tenant explicite.

3.22 Suivi en cours de sprint

Modalités de suivi. Le *burndown* réel est mis à jour quotidiennement au daily scrum. En fin de sprint, la rétrospective consigne : ce qui a bien fonctionné, ce qui peut être amélioré, et les actions pour le sprint suivant. Les fichiers éditables `SPRINT-0X.md` du dossier `roadmap/operationnel/02_sprints/` servent de support opérationnel ; ce document en est la vue consolidée.

CHAPITRE 4

Gestion des risques et de la charge

Ce chapitre consolide les risques identifiés sprint par sprint (chapitre 3) dans un **registre unique**, coté en probabilité $\times$ impact, et synthétise la charge estimée des plans d'exécution. Le registre est revu à chaque rétrospective (cf. §3.3.4).

4.1 Registre des risques

Cotation : probabilité et impact sur 3 niveaux (1 = faible, 2 = moyen, 3 = fort) ; criticité = probabilité $\times$ impact. Les risques de criticité ≥ 6 appellent une parade *préventive* (engagée avant le sprint concerné), les autres une parade *corrective* (déclenchée si le risque se matérialise).

ID	Risque	Sprint	P	I	C	Parade
R-01	Intégration PostGIS/Spring Boot plus complexe que prévu	S1	2	3	6	Spike jours 1-2 ; repli : lat/long NUMERIC, PostGIS différé en S5.
R-02	Mapping JPA du patron <i>Composite</i> (ruche)	S2	2	2	4	Hiérarchie à trois tables + compteurs contraints si blocage.
R-03	Retrofit RTL arabe coûteux sur écrans existants	S2+	2	2	4	Test RTL dès S2 sur tous les écrans livrés ; gabarit RTL de référence.
R-04	Service Worker / IndexedDB / synchronisation différée	S4	3	3	9	Spike en S3 ; périmètre limité à la saisie ; conflits dégradables en signalement manuel.
R-05	Configuration OAuth (console Google, redirections, secrets)	S4	2	2	4	Repli auth locale (US-021) livré dans le même sprint ; secrets en place avant S4.
R-06	Performance des agrégations SQL des dashboards	S5	2	2	4	Vues matérialisées ; index posés avec les vues ; mesure k6 en S7.

ID	Risque	Sprint	P	I	C	Parade
R-07	Charge de sprint supérieure à la vélocité réelle	tous	2	2	4	Rééquilibrage appliqué (§3.3) : amplitude ramenée de 26–57 à 36–39 pts, sous la capacité de référence de 40. Risque résiduel : cette capacité reste une hypothèse, à recalibrer sur la vélocité mesurée après S2. Fusibles désignés : US-019 (S4), US-029 (S6), US-033 (S7).
R-08	Broker MQTT et idempotence de l'ingestion	S6	2	2	4	Voie REST livrée d'abord ; MQTT en adaptateur ; idempotence testée avant source réelle.
R-09	Calibrage EWMA (S7) et extraction en microservice Python (S8) non aboutis	S7/S8	2	2	4	Découplage par contrat REST ; jeu de données simulé pour le calibrage ; l'application retombe sur le seuil simple de S5.
R-10	Couverture de tests insuffisante en fin de projet	S8	2	3	6	Quality gate progressif (50 % → 70 %) ; couverture ≈ 65 % exigée fin S7.
R-11	Documentation/UML concentrés sur le sprint final	S8	3	2	6	UML au fil de l'eau dès S2 ; ADR réutilisés dans le rapport ; <i>feature freeze</i> J10.
R-12	Dérive de périmètre (fonctions « inspiration marché »)	S7	2	2	4	Priorités MoSCoW gelées au sprint planning ; tout ajout passe par le Product Owner.
R-13	Indisponibilité ponctuelle de l'équipe (période académique)	tous	2	2	4	Capacité planifiée à 90 % (option D) ; stories fusibles par sprint.

Lecture. Trois risques dominent le projet : **R-04** (hors-ligne, criticité 9), **R-07** (surcharge S5/S7) et le couple **R-10/R-11** (qualité et documentation de fin de projet). Les trois sont traités *en amont* : spike S3, options d'équilibrage, quality gate progressif et UML au fil de l'eau.

4.2 Charge estimée par sprint

Synthèse des plans d'exécution du chapitre 3, en jours-homme (j-h) pour 10 jours ouvrés par sprint.

Périmètre de ce tableau : le plan initial à 8 sprints, avant rééquilibrage et avant l'extension à 18 sprints (chapitre 3, §3.3). Il est conservé parce que c'est lui qui a servi à coter le risque **R-07** (surcharge de S5 et S7) et à justifier le passage de la capacité de référence à 40 points — le remplacer par les chiffres d'aujourd'hui effacerait la raison d'être de cette cotation. Les totaux à jour — 18 sprints, 582 points, 540 j-h — sont au chapitre 3.

#	Sprint	Points	Charge (j-h)
1	Fondation — CRUD Core	31	14
2	Ruche & composition	26	12,5
3	Visites & rapports	31	14,5
4	Hors-ligne & synchronisation	26	12
5	Tableaux de bord	50	13
6	Capteurs & API	39	11,5
7	Fonctions avancées	57	13
8	Qualité & livraison	44	14
Total		304	104,5

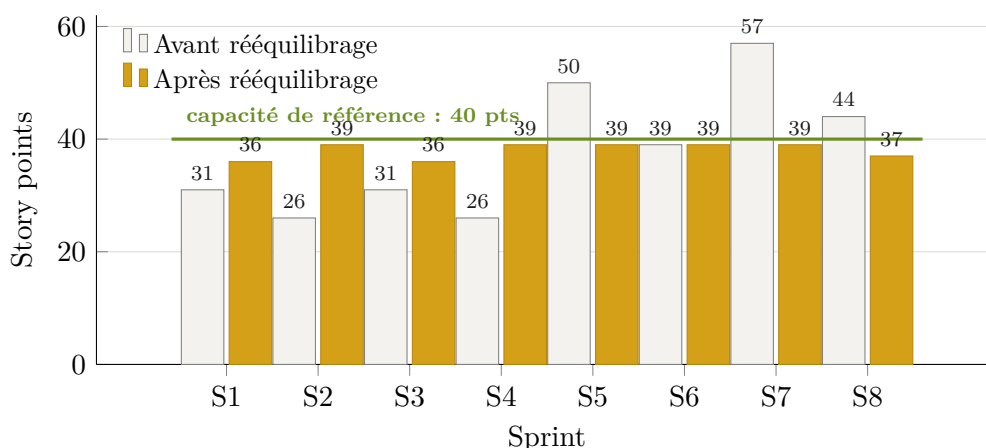


Figure 4.1. Points planifiés par sprint, avant et après rééquilibrage : la charge passe d’une amplitude de 26–57 points à 36–39, sous la capacité de référence (cf. registre R-07).

Dimensionnement de l’équipe. Les plans d’exécution (chapitre 3) sont chiffrés à **30 j-h par sprint**, soit **trois développeurs à temps plein** sur 10 jours ouvrés : environ 26,5 j-h de tâches techniques et 3,5 j-h de cérémonies Scrum et de revues de code, soit **540 j-h** sur les 18 sprints de construction. Le ratio obtenu, $\approx 0,93$ j-h par story point, suppose une équipe déjà à l’aise avec la pile Spring Boot ; pour une équipe qui la découvre, compter plutôt 1,2 j-h par point — soit ≈ 700 j-h au total, ou ≈ 39 j-h par sprint, que trois personnes ne couvrent pas. C’est l’hypothèse la plus sensible du plan et la première à recalibrer après S1 et S2 (cf. R-07 et *Suivi de la vélocité* ci-dessous). L’hypothèse de réutilisation — module photo, seuils, configuration — doit par ailleurs se vérifier dès S3.

4.3 Suivi de la vélocité

- **Après chaque sprint**, reporter la vélocité réelle (points terminés au sens de la DoD, pas « presque finis ») dans `roadmap/operationnel/02_sprints/sprints.json` ;
- **Après S2**, recalculer la capacité des sprints restants sur la moyenne glissante des deux derniers sprints, et arbitrer les options A/B du §3.3 ;
- **Signal d’alerte** : deux sprints consécutifs sous 80 % de l’engagement déclenchent une replanification des releases (chapitre 6) avec le Product Owner, plutôt qu’une accumulation silencieuse de reste-à-faire.

CHAPITRE 5

Pipeline DevOps (CI/CD)

La chaîne CI/CD s'appuie sur **GitHub Actions** (ou GitLab CI au choix) et se déclenche à chaque *push* ou *pull request*.

5.1 Environnements

Environnement	Branche	URL	Usage
Local	feature/*	http://localhost:8080/zumm	Développement local (Spring Boot)
Dev	develop	https://dev.zumm.local	Intégration continue, tests
Staging	release/*	https://staging.zumm.local	Recette, démo sprint review
Production	main	https://zumm.local	Déploiement manuel validé

5.2 Vue d'ensemble du pipeline

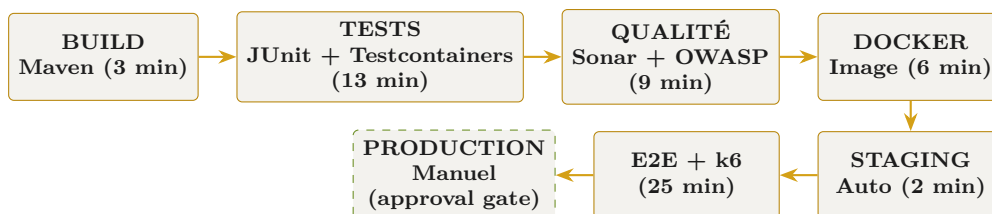


Figure 5.1. Enchaînement des étapes du pipeline CI/CD (durée totale ≈ 1 h).

5.3 Détail des étapes

Étape	Description	Commandes	Seuils / conditions	Durée
Build	Compilation Maven + packaging JAR autoporteur (JDK 17, Maven 3.9+, Spring Boot 3)	<code>mvn clean compile</code> <code>mvn package -DskipTests</code>	—	3 min
Tests unitaires	JUnit 5 + Mockito + JaCoCo	<code>mvn test</code> <code>mvn jacoco:report</code>	couverture $\geq 70\%$, 0 échec	5 min
Tests d'intégration	Testcontainers : PostgreSQL réel (PostGIS + TimescaleDB)	<code>mvn verify -P integration-tests</code>	—	8 min
Analyse qualité	SonarQube + SpotBugs + Checkstyle	<code>mvn sonar:sonar</code> <code>mvn spotbugs:check</code> <code>mvn checkstyle:check</code>	0 bug, 0 vulnérabilité, < 50 <i>code smells</i>	5 min
Sécurité	Scan des dépendances OWASP	<code>mvn dependency-check:check</code>	—	4 min
Build Docker	Image multi-étapes (build Maven puis JRE + JAR Zümm)	<code>docker build -t zumm:\${VERSION} .</code>	—	6 min
Déploiement staging	Déploiement automatique	<code>docker-compose -f docker-compose.staging.yml up -d</code>	branches <code>release/*</code> , <code>main</code>	2 min
Tests E2E	Selenium / Cypress sur staging	<code>mvn test -P e2e</code>	—	10 min
Tests de charge	k6, performance et robustesse	<code>k6 run tests/load/</code>	p95 < 500 ms, erreurs $< 1\%$	15 min
Déploiement production	Déploiement manuel (<i>approval gate</i>)	<code>docker-compose -f docker-compose.prod.yml up -d</code>	branche <code>main</code> + <i>ap-probation</i>	2 min

5.4 Infrastructure as Code

- **Docker** : `Dockerfile` applicatif multi-étapes (JAR Spring Boot), `Dockerfile` base de données (PostgreSQL + PostGIS), `docker-compose.yml` pour la stack complète (application + PostgreSQL + Nginx + monitoring) ;
- **Kubernetes** (optionnel, scaling futur) : manifestes `deployment.yml`, `service.yml`, `ingress.yml`, `configmap.yml`.

5.5 Mise en place progressive

Le pipeline complet décrit ci-dessus est la cible ; sa mise en place suit la montée en puissance du projet :

- **CI dès le sprint 1** (build + tests + Sonar), mais CD staging seulement à partir du sprint 2 : inutile de maintenir quatre environnements avant d’avoir une image conteneur stable ;
- **Quality gate progressif** : exiger 70 % de couverture dès le sprint 1 est irréaliste sur du code d’infrastructure — commencer à 50 % et monter de 5 points par sprint jusqu’à la cible ;
- **Gestion des secrets** (GitHub Secrets / vault) intégrée au pipeline avant le sprint 4 : l’authentification OAuth (US-020) en dépend ;
- **Tests E2E et de charge** activés à partir du sprint 5, quand les parcours utilisateur sont stabilisés.

5.6 Stratégie de branches Git

Branche	Rôle
main	Production stable
develop	Intégration continue
release/*	Stabilisation d’une release
feature/*	Développement d’une fonctionnalité
hotfix/*	Correction urgente en production

Fichiers opérationnels. Les fichiers exécutables du pipeline (`github-actions.yml`, `Dockerfile`, `docker-compose.yml`) sont maintenus dans `roadmap/operationnel/03_devops_pipeline/` ; ce chapitre en décrit le contrat (étapes, seuils, conditions).

CHAPITRE 6

Plan de releases

Le versionnage suit **Semantic Versioning** (SemVer : MAJOR.MINOR.PATCH). Quatre releases jalonnent le projet, une à l'issue de chaque paire de sprints.

6.1 Synthèse

Version	Nom	Date	Sprints	Statut
v0.1.0	MVP — Fondation	25/08/2026	S1, S2	Livrée
v0.2.0	Visites & collaboration	22/09/2026	S3, S4	Livrée
v0.3.0	Analytics & dashboards	20/10/2026	S5, S6	Livrée
v1.0.0	Release Candidate	16/11/2026	S7, S8	Livrée ¹
v1.1.0	Exploitation & spatial	18/01/2027	S9–S11	Livrée
v1.2.0	Sécurité & PWA livrable	15/02/2027	S12, S13	Livrée
v1.3.0	Conformité miel	15/03/2027	S14, S15	Livrée
v1.4.0	Sessions serveur & portée	29/03/2027	S16	Livrée
v1.5.0	Dettes soldées	26/04/2027	S17, S18	Livrée

6.2 v0.1.0 — MVP : Fondation

Contenu	CRUD complet des entités métier ; architecture Spring Boot multi-niveaux ; composition de la ruche (corps/hausse/cadre) ; configuration <code>ConfigZumm.ini</code> ; i18n FR/EN/AR ; sécurité TLS 1.3 / X.509 ; harnais de tests d'intégration Testcontainers.
Critère d'acceptation	Application déployable en HTTPS avec données de test.
Statut	extbfLivrée.

1. Sous réserve : les livrables documentaires US-039 et US-040 restent à produire. Voir l'encadré en fin de chapitre.

6.3 v0.2.0 — Visites & collaboration

Contenu	Workflow planification/approbation de visite ; rapport de visite complet avec photos ; mode hors-ligne PWA ; authentification OAuth 2.0 et repli local ; contrôle d'accès RBAC complet.
Critère d'acceptation	Parcours utilisateur complet en ligne et hors-ligne.
Statut	extbfLivrée.

6.4 v0.3.0 — Analytics & dashboards

Contenu	Calendrier agents × ruches ; tableaux de bord production & alertes ; synthèse ROI ; rappels et export CSV/TXT ; ingestion des mesures capteurs ; contexte météo local ; service web API (REST/OpenAPI).
Critère d'acceptation	Dashboards fonctionnels avec données réelles.
Statut	extbfLivrée.

6.5 v1.0.0 — Release Candidate

Contenu	Fonctionnalités avancées (carte et rayons de butinage, suivi de reine, QR code de traçabilité) ; détection d'anomalie adaptative (IA) ; tests complets (unitaires/intégration/charge) ; documentation & maintenance ; monitoring & alerting.
Critère d'acceptation	Solution complète prête pour la production.
Statut	extbfLivrée sous réserve — voir l'encadré ci-dessous.

Réserve sur la v1.0.0. Le périmètre *applicatif* est livré, mais la ligne « documentation & maintenance » ne l'est pas : US-039 (diagrammes UML) et US-040 (rapport, poster, présentation) restent à produire, la charte académique de l'épreuve interdisant de les générer. **21 des 37 points du sprint 8 ne sont donc pas acquis**, et la version ne peut pas être déclarée livrée sans cette réserve.

6.6 Après la v1.0.0 — cinq versions de consolidation

Les sprints 9 à 18 n'ajoutent pas de version majeure : ils durcissent et soldent un produit déjà livrable. Ils se répartissent en cinq versions mineures, toutes **livrées**.

Version	Sprints	Contenu
v1.1.0	S9–S11	Notifications, rapport PDF, audit, prévision ; requêtes spatiales PostGIS ; socle de test du front
v1.2.0	S12–S13	Durcissement de la sécurité (29 vulnérabilités $\rightarrow$ 0) ; PWA déployée, graphiques et carte
v1.3.0	S14–S15	Conformité miel (directive UE 2024/1438), idempotence ; ressources de langue externalisées
v1.4.0	S16	BFF : plus aucun jeton dans le navigateur (ADR-006) ; autorisation horizontale (US-057)
v1.5.0	S17–S18	Dettes techniques soldées ; agrégats calculés en base ; parité de types client/contrat

Règle de livraison. Une release n'est publiée que si tous les critères de la Definition of Done (§1.2.4) sont satisfaits pour l'ensemble des stories concernées, et si le pipeline CI/CD (chapitre 5) est intégralement vert sur la branche **release/*** correspondante.

CHAPITRE 7

Monitoring & observabilité

L'observabilité repose sur **Prometheus** + **Grafana** (métriques), **ELK** ou **Loki** (logs), **AlertManager** (alertes) et **Jaeger** (tracing distribué).

7.1 Dashboards Grafana

7.1.1 Zümm — Vue d'ensemble (métier)

Widget	Type	Requête PromQL
Ruches actives	stat	<code>count(ruche_status{status='active'})</code>
Visites aujourd'hui	stat	<code>sum(visites_total{date='today'})</code>
Production miel (kg)	graph	<code>sum(production_miel_kg) by (site)</code>
Alertes actives	graph	<code>count(alertes_status{status='active'})</code>
Latence API (p95)	heatmap	<code>histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))</code>

7.1.2 Zümm — Performance (technique)

Widget	Type	Requête PromQL
CPU JVM	graph	<code>jvm_cpu_usage_percent</code>
Mémoire heap	graph	<code>jvm_memory_heap_used_bytes</code>
Connexions DB	graph	<code>jdbc_connections_active</code>
Requêtes HTTP/s	graph	<code>rate(http_requests_total[1m])</code>

7.1.3 Zümm — Sécurité

Widget	Type	Requête PromQL
Tentatives échouées	stat	<code>sum(auth_failures_total[1h])</code>
Erreurs 4xx/5xx	graph	<code>sum(rate(http_errors_total[5m])) by (status)</code>
Top 10 IP suspectes	table	<code>topk(10, rate(requests_by_ip[5m]))</code>

7.2 Règles d'alerte

Alerte	Condition	Sévérité	Action
Latence API élevée	p95 sur 5 min > 500 ms	Warning	Notifier Slack + escalade si persistant
Taux d'erreurs > 1 %	<code>http_errors</code> / <code>http_requests</code> > 0,01	Critical	PagerDuty + rollback automatique
Connexions DB épuisées	actives / max > 0,9	Critical	Scaler le pool + alerte ops
JVM heap > 85 %	heap utilisé / heap max > 0,85	Warning	Analyse mémoire + redémarrage planifié

7.3 Gestion des logs

- **Niveaux** : ERROR, WARN, INFO, DEBUG ;
- **Corrélation** : en-tête `X-Request-ID` propagé pour le tracing distribué ;
- **Rétention** : 30 jours en stockage chaud, 1 an en stockage froid (S3).

Boucle DevOps. Les métriques métier (production, visites, alertes sanitaires) sont exposées par l'application elle-même : les dashboards de supervision technique et les tableaux de bord fonctionnels du cahier des charges (§4.3) partagent ainsi la même source de vérité.